Լաբորատոր աշխատանք 8
Տպագրության պատվեր

Աշխատանքի նպատակը. C++ ծրագրավորման լեզվով օբյեկտկողմնորոշված ծրագրավորման (OOP) սկզբունքների, դասերի (classes), կոնստրուկտորների, մուտքային տվյալների վավերացման (validation), զանգվածների և հիմնական ալգորիթմների (որոնում, զտում, զանգվածի տարրերի մշակում) կիրառման գործնական հմտությունների ամրապնդում՝ տպագրական պատվերների (PrintOrder) հաշվառման և կառավարման ավտոմատացված համակարգի մշակման միջոցով:

Խնդրի պահանջ. Ստեղծել `PrintOrder` դաս հետևյալ բնութագրերով. Անհատական տվյալներ (Private դաշտեր), պատվերի կոդ (`code` - տեքստային), էջերի քանակ (`pages` - ամբողջ թիվ),մեկ էջի արժեք (`pricePerUnit` - իրական թիվ), զեղչի տոկոս (`discount` - իրական թիվ):
Մեթոդներ. Էջերի ավելացման (`addPages`) մեթոդ՝ ստուգելով, որ ավելացվող քանակը դրական է:
Արժեքի հաշվարկման մեթոդներ՝ մինչև զեղչը (`calculateCostBeforeDiscount`):

```
#include <iostream>
#include <string>
using namespace std;
class PrintOrder {
private:
    string code;
    int pages;
    double pricePerUnit;
    double discount;
public:
    PrintOrder() {
        code = "Unknown";
        pages = 1;
        pricePerUnit = 1.0;
        discount = 0.0;
    }
    PrintOrder(string c, int p, double price) {
        code = (c.empty() ? "Unknown" : c);
        if (p > 0) {
            pages = p;
        }
        else {
            pages = 1;
        }
        if (price > 0) {
            pricePerUnit = price;
        }
        else {
            pricePerUnit = 1.0;
        }
```

```
    discount = 0.0;
}
PrintOrder(string c, int p, double price, double disc) {
    code = (c.empty() ? "Unknown" : c);

    if (p > 0) {
        pages = p;
    }
    else {
        pages = 1;
    }
    if (price > 0) {
        pricePerUnit = price;
    }
    else {
        pricePerUnit = 1.0;
    }
    if (disc >= 0 && disc <= 30) {
        discount = disc;
    }
    else {
        discount = 0.0;
    }
}
bool addPages(int p) {
    if (p > 0) {
        pages += p;
        return true;
    }
    cout << "Error: Pages to add must be positive!" << endl;
    return false;
}
double calculateCostBeforeDiscount() const {
    return pages * pricePerUnit;
}
double calculateCostAfterDiscount() const {
    double totalBefore = calculateCostBeforeDiscount();
    return totalBefore * (1.0 - discount / 100.0);
}
void show() const {
    cout << "Code: " << code
        << " | Pages: " << pages
        << " | Price/Page: " << pricePerUnit
        << " | Discount: " << discount << "%"
        << " | Cost after discount: " << calculateCostAfterDiscount() << endl;
}
string getCode() const { return code; }
```

```
    int getPages() const { return pages; }
    double getPricePerUnit() const { return pricePerUnit; }
    double getDiscount() const { return discount; }
};
int main() {
    const int SIZE = 5;
    PrintOrder orders[SIZE];
    for (int i = 0; i < SIZE; ++i) {
        string c = "Unknown";
        int p;
        double price = 1.0, disc = 0.0;
        char choice;
        cout << "\nEnter details for order #" << i + 1 << ":" << endl;
        cout << "Do you know the code? (y/n): ";
        cin >> choice;
        if (choice == 'y' || choice == 'Y') {
            cout << "Code: ";
            cin >> c;
        }
        else {
            c = "Unknown";
        }
        do {
            cout << "Pages (> 0): ";
            cin >> p;
            if (p <= 0) {
                cout << "Error: Pages must be a positive number! Please try again." << endl;
            }
        } while (p <= 0);
        cout << "Price per page: ";
        cin >> price;
        cout << "Is there a discount different from 0? (y/n): ";
        cin >> choice;
        if (choice == 'y' || choice == 'Y') {
            do {
                cout << "Discount (0-30): ";
                cin >> disc;
                if (disc < 0 || disc > 30) {
                    cout << "Error: Discount must be between 0 and 30! Please try again." << endl;
                }
            } while (disc < 0 || disc > 30);
        }
        else {
            disc = 0.0;
        }
        orders[i] = PrintOrder(c, p, price, disc);
    }
    cout << "\n=== All Orders List ===" << endl;
```

```
    for (int i = 0; i < SIZE; ++i) {
        orders[i].show();
    }
    string searchCode;
    cout << "\nEnter the order code to add pages: ";
    cin >> searchCode;

    int foundIdx = -1;
    for (int i = 0; i < SIZE; ++i) {
        if (orders[i].getCode() == searchCode) {
            foundIdx = i;
            break;
        }
    }
    if (foundIdx != -1) {
        if (orders[foundIdx].getDiscount() != 0) {
            cout << "Info: This order has a discount (" << orders[foundIdx].getDiscount() << "%)."
<< endl;
        }
        else {
            cout << "Info: This order has no discount (0%)." << endl;
        }
        int addMore;
        do {
            cout << "Enter pages to add to order (> 0): ";
            cin >> addMore;
            if (addMore <= 0) {
                cout << "Error: Pages to add must be positive! Try again." << endl;
            }
        } while (addMore <= 0);
        orders[foundIdx].addPages(addMore);
        cout << "Updated order details:" << endl;
        orders[foundIdx].show();
    }
    else {
        cout << "Error: Order with code '" << searchCode << "' not found!" << endl;
    }
    double totalCostAll = 0.0;
    int mostExpensiveIdx = 0;
    double maxCostAfterDisc = orders[0].calculateCostAfterDiscount();
    for (int i = 0; i < SIZE; ++i) {
        double currentCost = orders[i].calculateCostAfterDiscount();
        totalCostAll += currentCost;
        if (currentCost > maxCostAfterDisc) {
            maxCostAfterDisc = currentCost;
            mostExpensiveIdx = i;
        }
    }
```

```
    cout << "\n=== Results ===" << endl;
    cout << "Total cost of all orders: " << totalCostAll << endl;
    cout << "Most expensive order after discount:" << endl;
    orders[mostExpensiveIdx].show();
    return 0;
}
```